



AI Sandboxes

① Smarter Models

- Ability to call tools
- RL training

② Smarter Agents

- Debug actions by running linux commands
- Backtrack, replan & work towards the goal with a sandbox at their disposal

③ Security

- Minimal permissions

Manus AI

→ Uses a sandbox

Can be useful for codegen tasks

Proxies

→ Code exec & CDA

→ Has backtracking

→ Snapshot & restore

Micro VM

→ Security

→ Used as runtime

Speed

→ Imp for sandboxes

→ < 7s

→ PR to reduce under 1s

Port forwarding

→ Public URL

Backtracking

→ Agents

Configurable with existing docker
tooling.

Proxies Rest server

⇒ Spawns & manages MicroVMs

Each sandbox has VNC server & code server

Port forwarding to expose VNC server

ELF

MCP

Python SDK

ldev/kvm $\Rightarrow$ Linux virtualization
stack

start, stop, delete, snapshot
 $\Rightarrow$ All available via API

Also allow file upload & download.

AI Hub

Linux Sandboxing

$\rightarrow$ Execution Model

Thread - smallest unit of executⁿ

task - struct' in each thread. LL connecting struct-threads

"Process" $\Rightarrow$ Multiple threads

Kernel $\Rightarrow$ Privileged access to hardware

Kernel (supervisor

mode. $\Rightarrow$ need special
ins to switch
modes

int 0x80 -

Containers

- ⇒ Package app defs with its core business logic
- ⇒ Run arbitrary user code
- ⇒ Linux ⇒ collection of name spaces on different resources

Container has a restricted view.
Host can look into state of container

Container cannot go upwards & look into host machine

This way tells containers they have own resources

cgroups → Used with namespaces
controls how much of a resource a container can access.

Eg. What % of mem,
% of CPU for this container

Security of containers

Containers run as native processes on top of kernel

If you have kernel vuln, malicious process can exploit it & become root, and then game over.

How to sandbox containers

- Jail containers ⇒ Restrict syscalls they can call
- CAP ⇒ Capabilities that a process

Eg. CAP- NET, CAP- SYS-ADMIN

- Seccomp filters
 - Used to restrict what syscalls a process can call
- 'Minijail' ⇒ Lib by Chrome OS

Suppose we need more security?

Containers → Virtualization?

- Can run arbitrary code
- Each VM has its own guest userspace and guest kernel
 - Doesn't directly attack host kernel
- How do VMs access resources?

Linux Virtualization

- VMM → Virtual Machine Monitor
- sits on top of /dev/kvm, a device on kernel that exposes virtualization stack of a
- VMM talks to KVM device to spawn the VM & manages emulated devices inside.

To client, VMM thread looks blocked.
But actually, inside the thread, the
actual hardware code is running.

- When VM needs serving it exits out
via VMEXITS & are serviced, &
resumed with VMRESUMED.
- VMEXITS & VMRESUMED are expensive.
We should try to minimize them.

MicroVMs vs Traditional VMs

- In Rust - Uses google crosvm.
- Mem safe implementation of virtualization
- VMM 'jail' the emulated devices
separately.
- MicroVMs are written in Rust & have
this 'jailing' aspect.

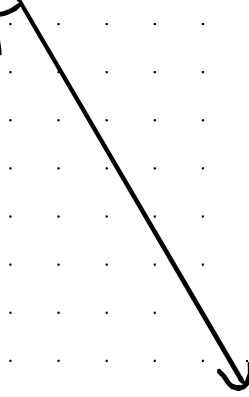
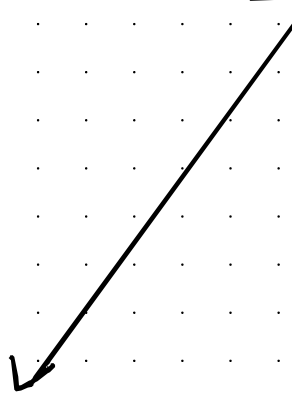
- Why micro?
- Boot fast + takes less memory
- QEMU is older, & supports more architecture
- Firecracker, etc. don't support many arch
 - This means that there's less codepaths
 - Leads to faster boot time & lesser memory consumption.

Aggrais : MicroVM Runtime

- Security is imp. Coding agents use case LLM generated code. Multi-tenant execution. Don't want unsafe code to read data of other tenants.
- Fast boot times
- Fast snapshotting → dump entire guest memory.

CrossVM
Base project

Desktop
Focused



Firecracker

cloud-hypervisor

- AWS Lambda
- Serverless loads
- Better REST api

- more general purpose VMM
- Hotplugging devices. Can add/remove RAM as a PCIe device
- GPU support
- Snapshot support
- This was used.

gvisor → closer to containers but better security.

Host server → Manage sandboxes

Analysis: Storage using Overlay FS

→ Problem: Malicious/buggy code that could delete code in sandboxes.

Could make sandbox non-functional

Eg: 'rm -rf /'

→ Need to protect roots running inside the sandbox.

→ Shared base layer shared btw sandboxes

→ Every sandbox gets its own read/write (RW) layer & read-only layer (RO)

→ When we snap shot a VM, only need to persist RW layer

How does this happen?

→ Setup Overlay FS as init script at VM boot time

→ This script will be PID 1.

→ Every process in sandbox just sees a regular file system.

Networking

→ Call other tools / API

→ Uses combo of:

→ Tap device: Each sandbox gets a unique tap device.

→ Linux bridge: On host. All taps connected.

→ Port forwarding: Forward ports to local Linux server.

VNC client can access GUI via VNC server in sandbox

Also lets customize what runs inside sandbox using Docker based tooling.

- Modify Dockerfile to add/remove packages
- Based on ubuntu
- standard packages present inside chrome also available

Creating VM

- Create devices
- OverlayFS
- Assign some memory
- Port forwarding
- VNC for comms
- spawn vmm

Snapshotting

- Agents generally fail at large tasks
- Need step by step plan
- shouldn't have to start from scratch if last step fails
- Lets agents save entire memory
- Agents can 'trackback' to good snapshot
- Zero overhead branching
- want to use Btrfs (Butter FS)
 - Natively aware of incremental snapshots
 - Looks like project is dead again.
- Persist RW OverlayFS

Ongoing work

→ Boot time < 18

→ Snapshots & persistence via BLOBs

→ Ballooning for memory management